

Guía de Docker

Instalación y Fundamentos para Desarrollo Web

Ing. Alex Javier Villegas Lainas

Sesión 2 (Material complementario) — 2026

Proyecto del ciclo: Sistema de Gestión de Encomiendas

¿Qué es Docker?

Docker es una plataforma de contenedorización que empaqueta una aplicación junto con todas sus dependencias en una unidad portátil llamada contenedor. Garantiza que el software funcione igual en cualquier entorno: tu laptop, el servidor del compañero, o producción en la nube.

PROBLEMA QUE RESUELVE DOCKER

Sin Docker: "Funciona en mi máquina pero no en la tuya" — distintas versiones de Python, PostgreSQL, librerías.

Con Docker: cada desarrollador del equipo corre exactamente el mismo ambiente, independientemente de su sistema operativo.

Conceptos clave

Componente	Descripción	Analogía
Dockerfile	Recetario para construir la imagen	Receta de cocina
Imagen	Plantilla inmutable de solo lectura	Molde de galleta
Contenedor	Instancia en ejecución de una imagen	Galleta hecha con el molde
Docker Hub	Registro público de imágenes	App Store de imágenes
Docker Compose	Orquesta múltiples contenedores	Director de orquesta

Docker vs Máquina Virtual

Ambas tecnologías aíslan entornos, pero lo hacen de manera muy distinta. Docker es la opción moderna para desarrollo y despliegue de aplicaciones web.

Aspecto	Docker (contenedor)	Máquina virtual
Sistema operativo	Comparte el kernel del host	SO completo virtualizado
Tiempo de arranque	Segundos	Minutos
Tamaño en disco	Megabytes	Gigabytes
Rendimiento	Casi nativo	Overhead significativo
Aislamiento	Proceso aislado	Completo (mayor seguridad)
Portabilidad	Muy alta (Docker Hub)	Media (imágenes pesadas)
Caso de uso ideal	Microservicios, CI/CD, dev	Diferentes SO en un host

CONCLUSIÓN

Para desarrollo web con Django, usa Docker. Para necesitar un Windows dentro de un Mac, usa una VM. No son excluyentes: muchas empresas corren Docker dentro de VMs en sus servidores.

Instalación de Docker

La forma recomendada es instalar Docker Desktop, que incluye: Docker Engine, Docker CLI, Docker Compose y un dashboard visual para gestionar imágenes y contenedores.

Windows 10 / 11

REQUISITO PREVIO

Windows 10 (64-bit, Build 19041+) o Windows 11 con WSL 2 habilitado.
WSL 2 es el backend que usa Docker Desktop en Windows.

Paso 1 — Activar WSL 2

Abre PowerShell como administrador y ejecuta:

PowerShell (Administrador)

```
wsl --install
```

```
# Verificar la instalación de WSL
```

```
wsl --list --verbose
```

```
# Si necesitas actualizar WSL
```

```
wsl --update
```

Reinicia el sistema si es la primera instalación de WSL.

Paso 2 — Descargar Docker Desktop

Ve a <https://www.docker.com/products/docker-desktop> y descarga el instalador para Windows.

Paso 3 — Instalar

1. Ejecuta el instalador descargado.
2. **Importante:** asegúrate de que “Use WSL 2 instead of Hyper-V” esté marcado.
3. Acepta los términos y haz clic en “Install”.
4. Reinicia el equipo cuando lo solicite.
5. Al reiniciar, Docker Desktop iniciará automáticamente (icono de ballena en la barra de tareas).

macOS

APPLE SILICON VS INTEL

Verifica tu chip en: Menú Apple → Acerca de esta Mac.
 Apple M1/M2/M3 → descarga la versión Apple Silicon (ARM64).
 Intel → descarga la versión Intel (x86_64).

Opción A — Docker Desktop (recomendado)

1. Ve a <https://www.docker.com/products/docker-desktop> y descarga la versión correcta.
2. Arrastra Docker.app a la carpeta Aplicaciones.
3. Abre Docker desde Aplicaciones y acepta los permisos.

Opción B — Homebrew**Terminal**

```
brew install --cask docker
```

Linux — Ubuntu 22.04 / 24.04**AVISO IMPORTANTE**

No instales Docker desde el repositorio oficial de Ubuntu (apt install docker.io).
 Usa el repositorio oficial de Docker para obtener la versión más reciente.

Terminal Ubuntu

```
# 1. Actualizar e instalar dependencias
sudo apt-get update
sudo apt-get install ca-certificates curl

# 2. Agregar la clave GPG oficial de Docker
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc

# 3. Agregar el repositorio oficial de Docker
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# 4. Instalar Docker Engine + Compose
```

```
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io \
    docker-buildx-plugin docker-compose-plugin
```

```
# 5. Agregar tu usuario al grupo docker (evita usar sudo)
```

```
sudo usermod -aG docker $USER
```

```
newgrp docker
```

Verificar la Instalación

Después de instalar, verifica que todo funciona correctamente. Si ves las versiones, ¡Docker está listo!

Terminal

```
# Versión de Docker Engine
docker --version
# -> Docker version 27.x.x, build xxxxxxxx

# Versión de Docker Compose
docker compose version
# -> Docker Compose version v2.x.x

# Información completa del sistema
docker info

# El clásico 'Hello World' de Docker
# Descarga la imagen, crea un contenedor y muestra el mensaje
docker run hello-world
```

¿QUÉ HACE DOCKER RUN HELLO-WORLD?

1. Docker busca la imagen hello-world localmente.
2. No la encuentra -> la descarga desde Docker Hub.
3. Crea un contenedor a partir de esa imagen.
4. Ejecuta el programa dentro del contenedor.
5. Muestra el mensaje de bienvenida.

Este es el ciclo completo de Docker: imagen -> contenedor -> ejecución.

Comandos de diagnóstico útiles

Terminal

```
# Ver imágenes descargadas localmente
docker images

# Ver contenedores en ejecución
docker ps

# Ver TODOS los contenedores (incluyendo detenidos)
docker ps -a
```

```
# Ver uso de CPU, memoria y red en tiempo real  
docker stats
```

```
# Ver cuánto espacio usa Docker en el sistema  
docker system df
```

Dockerfile

El Dockerfile es el recetario que le dice a Docker cómo construir la imagen de tu aplicación. Cada instrucción agrega una capa a la imagen.

Dockerfile
— INSTRUCCION 1: imagen base —————
FROM python:3.11-slim
'slim' = imagen minima sin herramientas extra (mas liviana)
— INSTRUCCION 2: variables de entorno —————
ENV PYTHONDONTWRITEBYTECODE=1 # no crear archivos .pyc
ENV PYTHONUNBUFFERED=1 # logs en tiempo real en la terminal
— INSTRUCCION 3: directorio de trabajo —————
WORKDIR /app
Todos los comandos siguientes se ejecutan desde /app
— INSTRUCCION 4: instalar dependencias —————
COPY requirements.txt .
RUN pip install --upgrade pip && pip install -r requirements.txt
Copiamos solo requirements.txt primero para aprovechar el cache
de capas. Si el codigo cambia pero no las deps, Docker no
reinstala todo -> builds mucho mas rapidos.
— INSTRUCCION 5: copiar el codigo fuente —————
COPY . .
— INSTRUCCION 6: exponer puerto —————
EXPOSE 8000
Documentacion: el contenedor escucha en el puerto 8000.
No lo publica automaticamente; eso lo hace -p en docker run.
— INSTRUCCION 7: comando de inicio —————
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]

Instrucciones principales del Dockerfile

Instrucción	Para qué sirve
FROM imagen:tag	Imagen base. Siempre la primera instrucción.
ENV CLAVE=valor	Define variables de entorno dentro del contenedor.

WORKDIR /ruta	Establece el directorio de trabajo para los siguientes comandos.
COPY origen destino	Copia archivos del host al sistema de archivos del contenedor.
RUN comando	Ejecuta comandos durante la construcción. Instala paquetes.
EXPOSE puerto	Documenta el puerto que usa el contenedor (no lo publica).
CMD ["cmd", "arg"]	Comando por defecto al iniciar el contenedor. Solo uno permitido.
ENTRYPOINT ["cmd"]	Punto de entrada fijo. CMD pasa argumentos a ENTRYPOINT.
ARG nombre	Argumento de construcción pasado con --build-arg.

Crear .dockerignore

Igual que .gitignore, evita copiar archivos innecesarios a la imagen. Reduce el tamaño y mejora la velocidad de build.

.dockerignore
.dockerignore
.env
venv/
env/
__pycache__/
*.pyc
.git/
db.sqlite3
*.log
.DS_Store

Comandos Esenciales

Imágenes

Comando	Descripción
<code>docker build -t nombre .</code>	Construir imagen desde Dockerfile en el directorio actual
<code>docker build -t nombre:tag .</code>	Construir con etiqueta de version especifica
<code>docker images</code>	Listar todas las imagenes locales
<code>docker pull imagen:tag</code>	Descargar imagen desde Docker Hub
<code>docker rmi imagen</code>	Eliminar una imagen local
<code>docker image prune</code>	Eliminar imagenes sin usar ("dangling")
<code>docker image prune -a</code>	Eliminar todas las imagenes no usadas por contenedores

Contenedores

Comando	Descripción
<code>docker run imagen</code>	Crear y arrancar un contenedor
<code>docker run -d imagen</code>	Correr en segundo plano (detached)
<code>docker run -p 8000:8000 img</code>	Mapear puerto host:contenedor
<code>docker run --name mi-app img</code>	Asignar nombre al contenedor
<code>docker run -v \$(pwd):/app img</code>	Montar directorio actual en /app del contenedor
<code>docker ps</code>	Ver contenedores en ejecucion
<code>docker ps -a</code>	Ver todos (incluyendo detenidos)
<code>docker stop id/nombre</code>	Detener contenedor con gracia (SIGTERM)
<code>docker start id/nombre</code>	Iniciar contenedor detenido
<code>docker restart id/nombre</code>	Reiniciar contenedor
<code>docker rm id/nombre</code>	Eliminar contenedor detenido
<code>docker rm -f id/nombre</code>	Forzar eliminacion aunque este corriendo

Inspección y depuración

Terminal
<code># Ver logs del contenedor</code>

```
docker logs nombre-contenedor

# Seguir logs en tiempo real (como tail -f)
docker logs -f nombre-contenedor

# Abrir una terminal DENTRO del contenedor
docker exec -it nombre-contenedor bash
# Si no tiene bash, usa sh:
docker exec -it nombre-contenedor sh

# Ver detalles completos (IP, volúmenes, variables, etc.)
docker inspect nombre-contenedor

# Copiar un archivo desde el contenedor al host
docker cp nombre-contenedor:/app/archivo.txt ./archivo.txt

# Ver uso de CPU, memoria y red en tiempo real
docker stats
```

Limpieza del sistema

Terminal

```
# Eliminar contenedores detenidos, imagenes dangling, redes sin uso
docker system prune

# Lo mismo pero tambien elimina imagenes no usadas por contenedores
docker system prune -a

# PELIGRO: tambien elimina volúmenes (borra datos de BD)
docker system prune -a --volumes

# Ver cuanto espacio usa Docker
docker system df
```

Volúmenes y Redes

Volúmenes

Los contenedores son efímeros: cuando se eliminan, sus datos desaparecen. Los volúmenes persisten los datos más allá del ciclo de vida del contenedor.

EJEMPLO PRÁCTICO EN EL PROYECTO

En el Sistema de Encomiendas, la base de datos PostgreSQL necesita un volumen para que los registros de envíos, clientes y rutas no se pierdan cada vez que se reinicia el contenedor.

Tipo	Sintaxis	Cuándo usar
Bind mount	-v ./local:/app	Desarrollo: hot-reload del código fuente
Volumen nombrado	-v nombre:/ruta	Producción: datos persistentes de BD
Volumen anónimo	-v /ruta	Datos temporales del contenedor

Terminal

```
# Crear un volumen nombrado (gestionado por Docker)
docker volume create postgres_data

# Usar el volumen con PostgreSQL
docker run -v postgres_data:/var/lib/postgresql/data postgres:15

# Bind mount: montar el código actual en /app (para hot-reload)
docker run -v $(pwd):/app -p 8000:8000 encomiendas

# Listar todos los volúmenes
docker volume ls

# Inspeccionar un volumen
docker volume inspect postgres_data

# Eliminar un volumen
docker volume rm postgres_data
```

Redes

Las redes permiten que contenedores se comuniquen entre sí. En Docker Compose, los servicios comparten automáticamente una red y pueden comunicarse usando el nombre del servicio como hostname.

Terminal

```
# Crear una red personalizada
docker network create encomiendas-net

# Correr PostgreSQL en esa red
docker run --network encomiendas-net --name db postgres:15

# Django puede conectarse usando 'db' como hostname
# (el nombre del contenedor se resuelve automaticamente a su IP)
docker run --network encomiendas-net --name web \
  -e DB_HOST=db encomiendas:v1

# En settings.py -> DB_HOST = 'db' (nombre del servicio)

# Listar redes
docker network ls

# Eliminar una red
docker network rm encomiendas-net
```

Docker Compose

Docker Compose orquesta múltiples contenedores con un solo archivo `docker-compose.yml`. Perfecto para aplicaciones con varios servicios: web + base de datos + cache.

docker-compose.yml	
version: '3.9'	
services:	
# — Servicio Django —————	
web:	
build: .	# construir desde Dockerfile local
command: python manage.py runserver 0.0.0.0:8000	
volumes:	
- ./app	# bind mount: hot-reload del código
ports:	
- "8000:8000"	
env_file:	
- .env	# variables de entorno desde .env
depends_on:	
- db	# espera que db este disponible
# — Servicio PostgreSQL —————	
db:	
image: postgres:15-alpine	# imagen oficial, variante ligera
volumes:	
- postgres_data:/var/lib/postgresql/data/	
environment:	
POSTGRES_DB: \${DB_NAME}	
POSTGRES_USER: \${DB_USER}	
POSTGRES_PASSWORD: \${DB_PASSWORD}	
volumes:	
postgres_data:	# volumen nombrado: datos persisten

Comandos de Docker Compose

Comando	Descripción
docker compose up	Levantar todos los servicios (foreground)
docker compose up -d	Levantar en segundo plano (detached)

<code>docker compose up --build</code>	Reconstruir imagenes antes de levantar
<code>docker compose up --build -d</code>	Build + levantar en segundo plano
<code>docker compose down</code>	Detener y eliminar contenedores
<code>docker compose down -v</code>	También elimina los volúmenes
<code>docker compose ps</code>	Estado de todos los servicios
<code>docker compose logs web</code>	Ver logs de un servicio específico
<code>docker compose logs -f</code>	Seguir logs en tiempo real
<code>docker compose exec web bash</code>	Abrir terminal en el servicio web
<code>docker compose exec web cmd</code>	Ejecutar comando en el contenedor
<code>docker compose restart web</code>	Reiniciar un servicio específico
<code>docker compose build</code>	Reconstruir solo las imagenes
<code>docker compose stop</code>	Detener servicios sin eliminarlos

Flujo diario en el proyecto

Terminal

```
# — Primer arranque del proyecto —————
docker compose up --build -d

# — Aplicar migraciones a PostgreSQL —————
docker compose exec web python manage.py makemigrations
docker compose exec web python manage.py migrate

# — Crear superusuario para el Admin —————
docker compose exec web python manage.py createsuperuser

# — Verificar en el navegador —————
# App:    http://localhost:8000
# Admin: http://localhost:8000/admin

# — Si algo falla, ver los logs —————
docker compose logs -f web
docker compose logs -f db

# — Al terminar el día —————
docker compose down
```

Docker Hub

Docker Hub es el registro público de imágenes Docker. Aquí encontrarás imágenes oficiales de Python, PostgreSQL, Redis, Nginx y miles más.

Terminal
Iniciar sesion en Docker Hub docker login
Descargar imagenes que usaremos en el proyecto docker pull python:3.11-slim docker pull postgres:15-alpine docker pull redis:7-alpine # para WebSockets (semana 6-7) docker pull nginx:alpine # para produccion (semana 8)
Buscar imagenes desde la terminal docker search postgres
Etiquetar tu imagen para publicar en Docker Hub docker tag encomiendas:v1 tuusuario/encomiendas:v1
Publicar en Docker Hub docker push tuusuario/encomiendas:v1

Imágenes oficiales del proyecto

Imagen	Descripción	Semana de uso
python:3.11-slim	Base de la app Django. Slim = minima, sin herramientas extra.	1-8
postgres:15-alpine	Base de datos del proyecto. Alpine = variante minima de Linux.	2-8
redis:7-alpine	Cache y channel layer para WebSockets en tiempo real.	6-7
nginx:alpine	Servidor web inverso para despliegue en produccion.	8

Cheatsheet — Referencia Rápida

Los comandos que usarás todos los días en el proyecto de encomiendas.

Cheatsheet completo	
# =====	
# BUILD	
# =====	
docker build -t app .	# construir imagen
docker compose up --build -d	# compose: build + levantar
# =====	
# RUN / STOP	
# =====	
docker compose up -d	# levantar servicios
docker compose down	# bajar servicios
docker compose down -v	# bajar + borrar volúmenes
docker compose restart web	# reiniciar solo el web
# =====	
# DEBUG	
# =====	
docker compose logs -f web	# logs del servicio web
docker compose logs -f db	# logs de PostgreSQL
docker compose exec web bash	# terminal dentro del contenedor
docker compose ps	# estado de todos los servicios
docker stats	# uso de recursos en tiempo real
# =====	
# DJANGO dentro del contenedor	
# =====	
docker compose exec web python manage.py migrate	
docker compose exec web python manage.py makemigrations	
docker compose exec web python manage.py createsuperuser	
docker compose exec web python manage.py shell	
docker compose exec web python manage.py collectstatic	
# =====	
# LIMPIEZA	
# =====	
docker system prune	# limpiar recursos sin usar
docker system prune -a	# limpiar todo lo no usado
docker system df	# ver uso de espacio

Checklist de Verificación

Completa estos pasos antes de la Sesión 3. Deberás mostrar el sistema corriendo al inicio de la clase.

☐	Docker Desktop instalado y el icono de ballena está activo
☐	<code>docker --version</code> muestra la versión correctamente
☐	<code>docker compose version</code> funciona en la terminal
☐	<code>docker run hello-world</code> ejecuta sin errores
☐	Proyecto Django tiene Dockerfile en la raíz
☐	<code>docker-compose.yml</code> creado con servicios web y db
☐	Archivo <code>.env</code> configurado con variables de la base de datos
☐	<code>.env</code> y <code>.dockerignore</code> están en <code>.gitignore</code>
☐	<code>docker compose up --build -d</code> levanta sin errores
☐	<code>docker compose exec web python manage.py migrate</code> funciona
☐	Panel admin accesible en <code>http://localhost:8000/admin</code>

LISTO PARA LA SESIÓN 3

Con Docker correctamente configurado, la Sesión 3 trabajaremos los Modelos ORM y las relaciones entre Encomienda, Cliente y Ruta directamente sobre esta base containerizada.